ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1-2

Разработка интерактивной HTML-страницы «Mercedes-AMG F1 — Центр производительности»

Дисциплина: Разработка интернет-приложений

Выполнил(а): _____Головастикова П.Я._

Группа: ____ИБМ3-54Б_____

Проверил(а): _____

2026

Содержание

1. Цель и задачи работы

2. Общая характеристика HTML-страницы

3. Структура документа HTML

4. Описание CSS-оформления

5. Разбор HTML-разметки страницы

6. Разбор JavaScript-кода

7. Адаптивность и взаимодействие с пользователем

8. Проверка работы и ограничения

9. Заключение

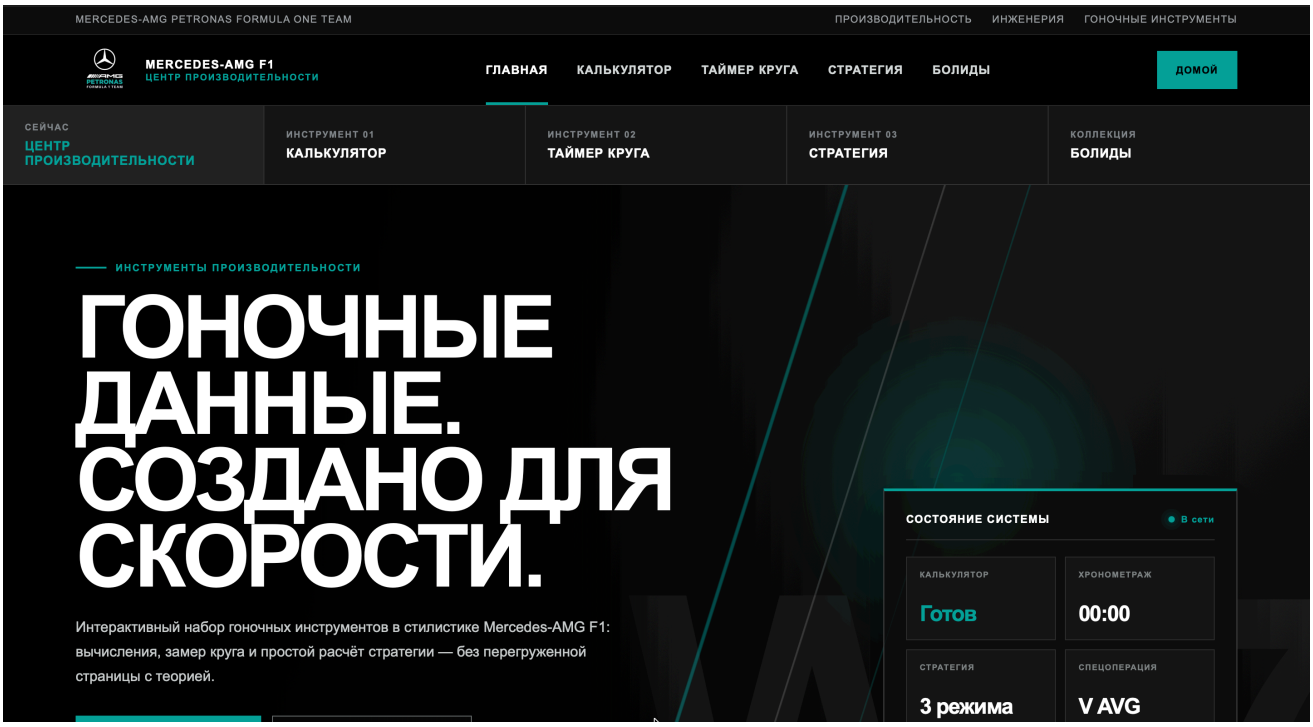
1. Цель и задачи работы

Цель лабораторной работы — изучить устройство одностраничного веб-интерфейса, созданного с применением HTML, CSS и JavaScript, и рассмотреть принципы организации разметки, визуального оформления и интерактивного поведения элементов.

В ходе работы необходимо:

- изучить структуру HTML-документа и назначение основных семантических элементов;
- рассмотреть применение CSS-переменных, Flexbox, CSS Grid, псевдоэлементов и медиазапросов;
- разобрать механизм переключения экранов без загрузки отдельных HTML-файлов;
- описать логику калькулятора, таймера круга и демонстрационного расчёта гоночной стратегии;
- определить, какие действия пользователя вызывают обработчики JavaScript и изменяют страницу;
- оценить адаптивность, ограничения и возможные направления дальнейшего развития.

2. Общая характеристика HTML-страницы



Исходный файл Mercedes_AMG_F1_Performance_Hub_RU.html представляет собой самостоятельную веб-страницу на русском языке. В одном документе объединены HTML-разметка, CSS-стили и JavaScript-логика. Внешние библиотеки для основных функций не подключаются: поведение реализовано средствами стандартного JavaScript и браузерных API.

Интерфейс стилизован под гоночную тематику Mercedes-AMG PETRONAS Formula One Team. Используются чёрный и тёмно-серый фон, серебристые надписи и бирюзовый акцент. Страница организована как мини-приложение с четырьмя экранами:

- Главная — вводный экран с описанием инструментов и карточками перехода.
- Калькулятор — арифметические действия, проценты, изменение знака, удаление символа, история вычислений и специальный расчёт средней скорости.
- Таймер круга — запуск и остановка секундомера, фиксация отдельных кругов, поиск лучшего времени.
- Гоночная стратегия — оценка топлива, числа пит-стопов, примерного окна остановки и суммарного времени на основе введённых параметров.

Важно: страница является демонстрационным интерфейсом. В самом коде прямо указано, что расчёт шинной стратегии построен на простых условных порогах и не является настоящей стратегической моделью Formula 1.

3. Структура документа HTML

3.1. Объявление типа документа и корневой элемент

```
<!DOCTYPE html>
<html lang="ru">
```

DOCTYPE сообщает браузеру, что документ использует современный стандарт HTML. Атрибут lang="ru" указывает основной язык содержимого — русский. Это полезно для средств доступности, поисковых систем и автоматического определения языка.

3.2. Секция head

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Mercedes-AMG F1 — Центр производительности</title>
```

В head задаются метаданные и заголовок вкладки браузера. UTF-8 обеспечивает корректное отображение кириллицы. viewport настраивает масштабирование страницы на мобильных устройствах. Внутри head также расположен большой блок style со всеми CSS-правилами.

3.3. Секция body и общий контейнер

```
<body>
  <div class="app"> ... </div>
</body>
```

В body находится видимое содержимое. Контейнер .app объединяет интерфейс и задаёт общий фон. Внутри него размещены верхняя информационная полоса, шапка, панель быстрого выбора инструмента, основная область main и подвал footer.

3.4. Семантические элементы

- header — верхняя часть сайта с брендингом, навигацией и кнопкой меню.
- nav — блок навигационных ссылок.
- main — основное содержимое приложения.
- section — логически самостоятельные экраны и смысловые разделы.
- article — самостоятельные карточки инструментов и информационные блоки.
- aside — вспомогательные панели, например история вычислений или список кругов.

- footer — нижняя часть страницы с названием проекта.

Семантическая разметка делает структуру понятнее и облегчает поддержку проекта.

4. Описание CSS-оформления

4.1. CSS-переменные :root

```
:root {
  --black: #000000;
  --panel: #211e1b;
  --silver: #c8ccce;
  --teal: #00a19b;
  --white: #ffffff;
  --page: 1280px;
  --header: 78px;
}
```

В :root определяются пользовательские свойства CSS. Благодаря им цветовая схема, максимальная ширина контента и высота шапки задаются централизованно. Например, var(--teal) используется для активных ссылок, рамок, кнопок и выделения значений. Это упрощает изменение темы сайта.

4.2. Сброс и базовые стили

```
* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body { margin: 0; min-height: 100vh; ... }
a { color: inherit; text-decoration: none; }
```

Универсальный селектор устанавливает предсказуемую модель расчёта размеров. Удаление стандартного внешнего отступа body помогает контролировать компоновку. Плавная прокрутка улучшает визуальное восприятие, а стили ссылок убирают стандартное подчёркивание и наследуют цвет текста.

4.3. Flexbox и CSS Grid

Flexbox применяется там, где элементы нужно выстроить в одну строку или распределить по краям: например, в верхней полосе, шапке, кнопках и подвале. CSS Grid используется для более структурированных сеток: четырёх вкладок инструментов, карточек на главной, клавиатуры калькулятора, формы стратегии и блоков результатов.

```
display: flex;
justify-content: space-between;
align-items: center;

 display: grid;
 grid-template-columns: repeat(4, 1fr);
```

Единица fr распределяет доступное пространство между колонками. Функции minmax() и repeat() позволяют строить гибкие сетки и избегать слишком узких колонок.

4.4. Фоновые эффекты, переходы и псевдоэлементы

Для фона используются linear-gradient() и radial-gradient(), создающие затемнение, световые пятна и акценты. Переходы transition смягчают изменение цвета, рамок и положения при наведении. Псевдоэлементы ::before и ::after добавляют декоративные элементы без дополнительных узлов HTML: линию активного пункта меню, номера карточек, декоративный текст W17 и световые полосы.

Свойства cursor, transform, box-shadow, border и backdrop-filter используются для создания ощущения панели гоночного оборудования. Декоративные элементы, не предназначенные для взаимодействия, часто получают pointer-events: none.

4.5. Медиазапросы и адаптивность

```
@media (max-width: 980px) { ... }
@media (max-width: 650px) { ... }
```

При ширине экрана до 980 пикселей скрывается обычная навигация, появляется кнопка мобильного меню, карточки и рабочие области перестраиваются в одну колонку. При ширине до 650 пикселей скрывается информационная полоса, уменьшаются отступы и размеры отдельных элементов, а кнопки становятся широкими и удобными для касания. Это позволяет использовать интерфейс на компьютерах, планшетах и смартфонах.

5. Разбор HTML-разметки страницы

5.1. Верхняя информационная полоса и шапка

Блок .utility-bar содержит название команды и краткие тематические подписи. Элемент header.site-header закрепляется в верхней части окна с помощью position: sticky и top: 0. Свойство z-index помогает сохранять шапку над содержимым при прокрутке.

В блоке .brand размещены встроенное изображение логотипа и текстовое название проекта. Навигационные ссылки используют href вида #/home и атрибут data-route. Кнопка mobileToggle управляет раскрытием меню на небольших экранах.

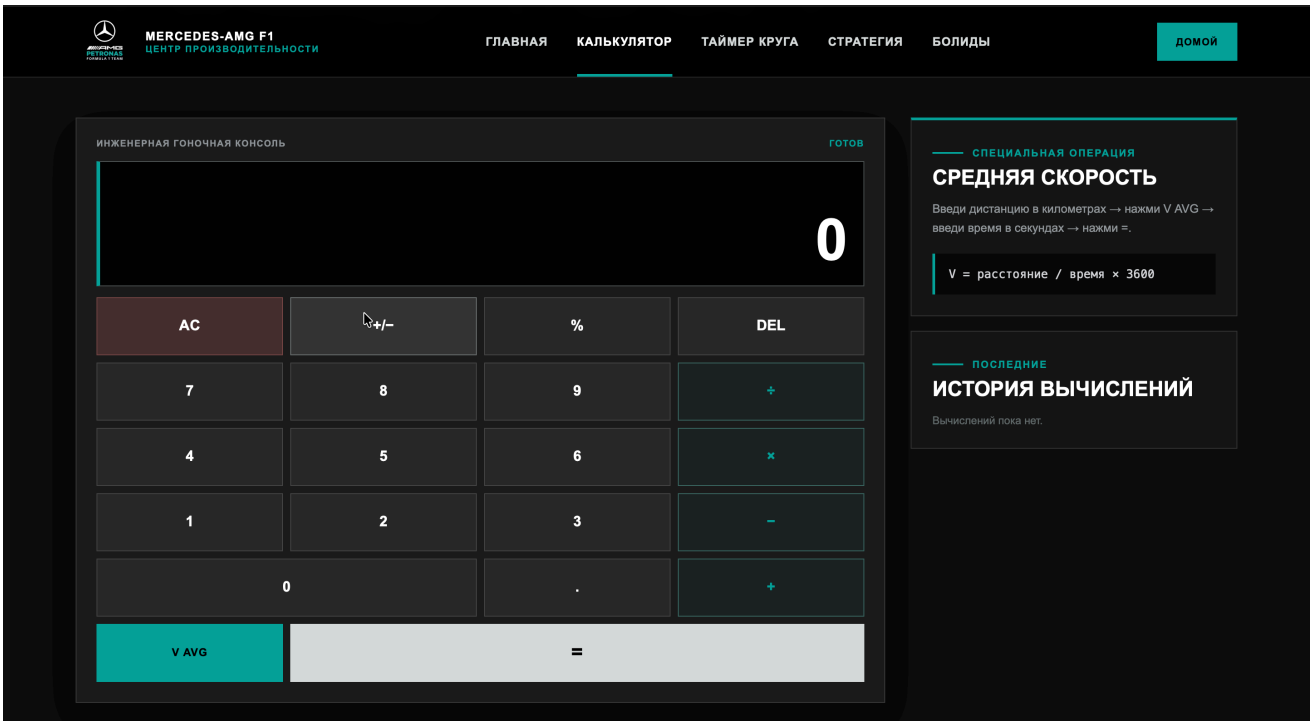
5.2. Панель быстрого выбора

Элемент .tool-strip содержит четыре вкладки. Каждая вкладка имеет подпись small и название strong. Атрибут data-route связывает её с соответствующим экраном. CSS-класс active обозначает текущий выбранный раздел.

5.3. Главный экран

Экран с атрибутом data-page="home" включает hero-блок и сетку карточек. В hero размещены заголовок, описание и кнопки перехода. Справа находится стилизованная консоль с демонстрационными показателями. Карточки .feature-card кратко объясняют назначение калькулятора, таймера и стратегического расчёта.

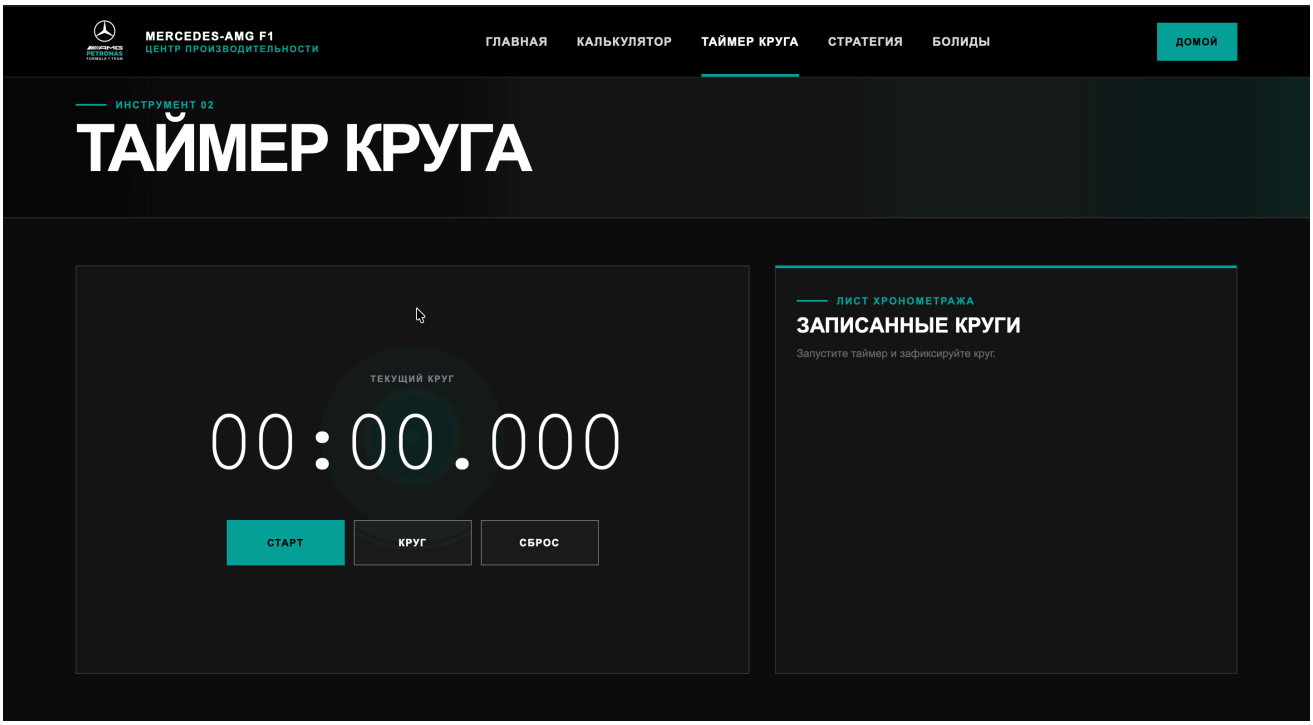
5.4. Экран калькулятора



Калькулятор разделён на основную панель .calc-panel и боковые информационные карточки. Область calcDisplay показывает текущий ввод или результат, calcExpression — выражение, calcStatus — состояние операции. Кнопки используют идентификаторы и data-атрибуты, по которым JavaScript назначает обработчики.

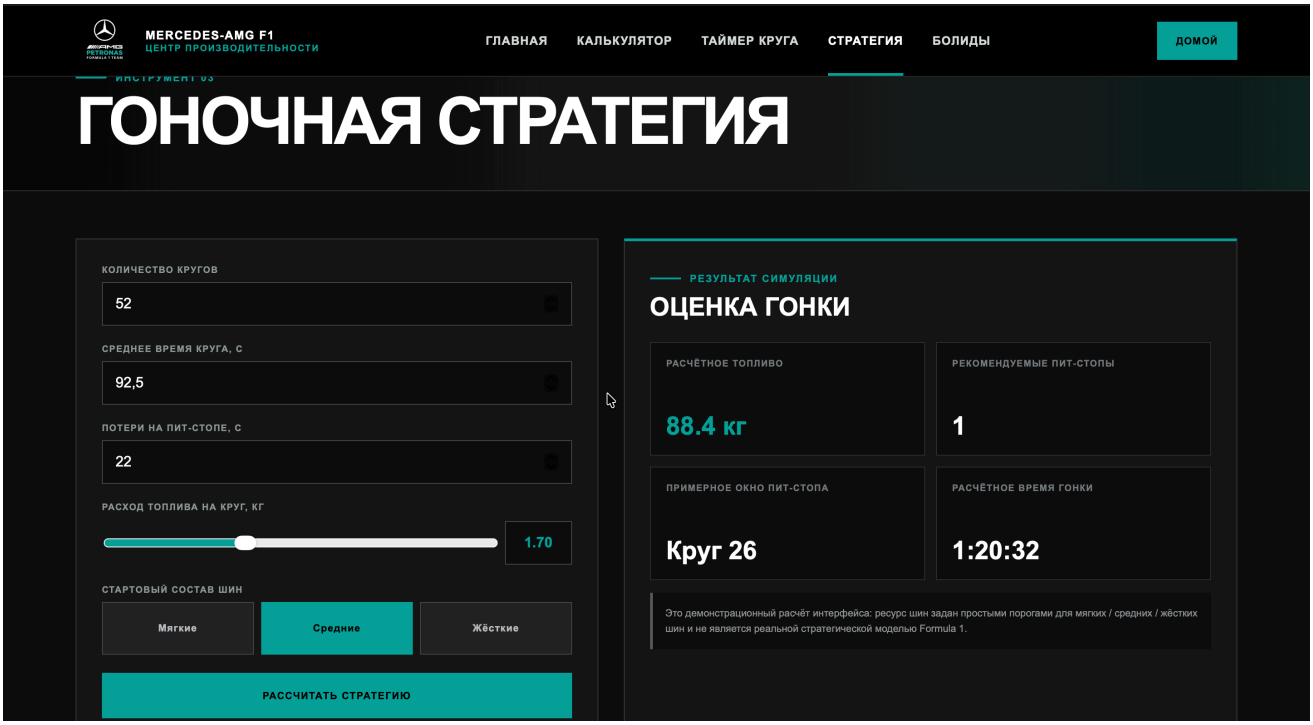
Клавиатура организована сеткой из четырёх колонок. Специальная кнопка V AVG запускает режим вычисления средней скорости по формуле V = расстояние / время × 3600, если расстояние введено в километрах, а время — в секундах.

5.5. Экран таймера круга



Блок .timer-board содержит отображение времени и три управляющие кнопки: «Старт», «Круг» и «Сброс». В боковой панели lapList выводятся зафиксированные круги. Строки результатов создаются динамически с помощью JavaScript.

5.6. Экран гоночной стратегии



Форма стратегии включает число кругов, среднее время круга, потери времени на пит-стопе, ползунок расхода топлива и выбор состава шин. Справа расположены четыре результата: топливо, количество остановок, примерное окно пит-стопа и расчётное время гонки.

Элементы формы имеют уникальные id, благодаря которым сценарий получает их значения через document.getElementById(). Результаты обновляются без перезагрузки документа.

6. Разбор JavaScript-кода

6.1. Получение элементов страницы

```
const pages = document.querySelectorAll('.page');
const routeLinks = document.querySelectorAll('[data-route]');
const mainNav = document.getElementById('mainNav');
```

Методы querySelectorAll() и getElementById() находят элементы DOM — объектной модели документа. Переменные const хранят ссылки на найденные узлы. let используется там, где значение будет изменяться в ходе работы, например в состоянии калькулятора или таймера.

6.2. Переключение экранов

```
function openPage(pageName) {
  const allowedPages = ['home', 'calculator', 'timing', 'strategy'];
  if (!allowedPages.includes(pageName)) pageName = 'home';
  pages.forEach(function(page) {
    page.classList.toggle('active', page.dataset.page === pageName);
  });
}
```

Функция openPage() принимает имя экрана, проверяет его по списку допустимых значений и назначает класс active только нужному разделу. Остальные страницы скрываются CSS-правилом .page { display: none; }. Затем активное состояние синхронизируется с навигацией, мобильное меню закрывается, а прокрутка возвращается наверх.

Функция routeFromHash() читает часть URL после символа #/. Событие hashchange позволяет переключать экран при изменении хеша. Такой подход имитирует простую маршрутизацию одностраничного приложения, но не загружает отдельные HTML-документы.

6.3. Состояние калькулятора

Для калькулятора используются следующие основные переменные:

- displayValue — строка, отображаемая на экране калькулятора;
- firstValue — первое числовое значение операции;
- selectedOperation — выбранная операция (+, −, ×, ÷ или avg);
- waitingForSecond — флаг ожидания ввода второго числа;
- calculationHistory — массив последних вычислений.

6.4. Ввод и форматирование чисел

Функция formatNumber() проверяет, является ли результат конечным числом, округляет его до восьми знаков после запятой и преобразует в строку. При недопустимом числовом результате

возвращается текст «Ошибка». Функция inputDigit() добавляет цифру к текущему значению, но ограничивает длину ввода шестнадцатью символами. Если ожидается второе число, ввод начинает новое значение.

inputDecimal() добавляет десятичную точку только в том случае, если её ещё нет. При начале второго числа отображение становится «0.». chooseOperation() сохраняет первое число, записывает выбранную операцию и переводит калькулятор в режим ожидания второго операнда.

6.5. Арифметические вычисления

```
function calculateValues(a, b, operation) {
  switch (operation) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return b === 0 ? null : a / b;
    case 'avg': return b === 0 ? null : a / b * 3600;
    default: return null;
  }
}
```

Функция calculateValues() выбирает действие через switch. Деление на ноль предотвращается проверкой b === 0. В режиме avg первое значение трактуется как дистанция в километрах, второе — как время в секундах; умножение на 3600 переводит километры в секунду в километры в час.

handleEquals() проверяет, введены ли оба операнда и выбрана ли операция. Если результат некорректен, пользователь видит сообщение об ошибке. В противном случае выражение и результат отображаются на экране, операция записывается в историю, а рабочие переменные сбрасываются для следующего вычисления.

6.6. Дополнительные функции калькулятора

- handleClear() очищает текущее вычисление и возвращает начальное состояние.
- handleBackspace() удаляет последний символ текущего числа.
- toggleSign() меняет знак числа на противоположный.
- makePercent() делит текущее число на 100.
- addHistory() добавляет запись в начало массива истории и оставляет не более шести последних результатов.

Для отображения истории контейнер очищается и заполняется заново элементами div. Текст записи задаётся через textContent, что безопаснее для обычного текстового содержимого, чем интерпретация строки как HTML.

6.7. Управление с клавиатуры

Общий обработчик keydown позволяет вводить цифры и операции с физической клавиатуры. Сначала код проверяет, не находится ли фокус в input, textarea или select, а также активен ли экран калькулятора. Enter и = выполняют вычисление, Backspace и Delete удаляют символ, Escape сбрасывает калькулятор. После обработки event.preventDefault() предотвращает

стандартное действие браузера, а flashKeyboardButton() на короткое время добавляет кнопке класс визуальной активности.

6.8. Таймер круга

Таймер хранит идентификатор интервала timerInterval, момент старта timerStartedAt, прошедшее время timerElapsed и массив lapTimes. Для измерения используется performance.now(), который возвращает высокоточное значение времени. setInterval() обновляет показания примерно каждые 25 миллисекунд.

Функция startStopTimer() запускает таймер, если интервал ещё не создан, либо останавливает его через clearInterval(). При повторном запуске начальная отметка вычисляется с учётом уже прошедшего времени, поэтому пауза не обнуляет отсчёт.

formatTime() преобразует миллисекунды в формат минуты:секунды:миллисекунды. recordLap() записывает суммарное время и длительность текущего круга как разность текущего общего времени и предыдущей отметки. renderLaps() определяет минимальную длительность круга и помечает соответствующую строку классом best. resetTimer() останавливает интервал, очищает результаты и возвращает интерфейс в исходное состояние.

6.9. Расчёт гоночной стратегии

Функция tyreLife() возвращает условный ресурс комплекта: 18 кругов для мягких шин, 26 для средних и 36 для жёстких. Эти значения являются упрощёнными демонстрационными параметрами.

```
const stops = Math.max(0, Math.ceil(laps / stintLife) - 1);
const pitWindow = stops === 0
  ? 'Без пит-стопа'
  : 'Круг ' + Math.round(laps / (stops + 1));
const fuel = laps * fuelPerLap;
const raceTime = laps * averageLap + stops * pitSeconds;
```

calculateStrategy() считывает значения формы и ограничивает минимальные значения кругов, времени и потерь на пит-стопе. Количество остановок рассчитывается по условному ресурсу шин. Топливо равно числу кругов, умноженному на расход топлива за круг. Время гонки рассчитывается как сумма времени всех кругов и времени, добавленного остановками. Результаты выводятся в DOM.

Ползунок топлива обновляет подпись и запускает перерасчёт. Кнопки выбора шин меняют selectedCompound и визуальный класс active. Поля формы пересчитывают результат при изменении значений, а кнопка «Рассчитать стратегию» запускает тот же расчёт вручную.

6.10. Обработчики событий и обновление интерфейса

В коде активно используются onclick, oninput, addEventListener() и обработчики событий window. Они связывают действия пользователя с функциями приложения. Например, нажатие кнопки запускает функцию, изменение ползунка пересчитывает стратегию, а изменение хеша URL переключает экран. Изменения текста выполняются через textContent, а строки истории и списка кругов создаются динамически.

7. Адаптивность и взаимодействие с пользователем

Интерфейс предусматривает несколько уровней обратной связи: активный пункт меню, подсветка кнопок, статус калькулятора, выделение лучшего круга, обновление результатов стратегии и мобильное раскрывающееся меню. Анимации применяются умеренно и в основном связаны с появлением экранов и состояниями наведения.

Страница использует hash-навигацию, поэтому выбранный раздел отражается в адресе после символа #. При прямом открытии адреса с известным хешем функция routeFromHash() выбирает соответствующий экран. Неизвестный маршрут возвращает пользователя на главную страницу.

8. Проверка работы и ограничения

8.1. Рекомендуемые проверки

1. Открыть HTML-файл в современном браузере и проверить загрузку главного экрана.
2. Перейти во все четыре раздела через верхнее меню, панель инструментов и карточки главной страницы.
3. В калькуляторе проверить сложение, вычитание, умножение, деление, деление на ноль, проценты, смену знака и очистку.
4. Проверить ввод с клавиатуры и убедиться, что он не перехватывается в полях формы.
5. Запустить таймер, остановить его, продолжить, записать несколько кругов и выполнить сброс.
6. Изменить количество кругов, время круга, потери на пит-стопе, расход топлива и состав шин; проверить обновление результатов.
7. Изменить ширину окна и проверить меню и расположение карточек на мобильном размере.

8.2. Ограничения и замечания

- Расчёт стратегии учебный и не учитывает реальные данные трассы, деградацию шин, погоду, трафик, правила соревнований и множество других факторов.
- Таймер работает в открытой вкладке браузера; это не специализированный измерительный прибор. Частота обновления интерфейса не означает, что обновление экрана происходит ровно каждые 25 миллисекунд.
- История калькулятора хранится в оперативной памяти и очищается при перезагрузке страницы.
- Результаты таймера также хранятся только в текущем состоянии страницы и не сохраняются в файл или базу данных.
- Встроенное изображение логотипа задано через data URI, поэтому отдельный файл изображения для этого элемента не требуется.
- Следует отдельно проверить доступность интерфейса с клавиатуры и экранными дикторами; наличие aria-label только у кнопки мобильного меню не означает полной проверки доступности.

9. Заключение

В ходе лабораторной работы рассмотрена структура интерактивной веб-страницы, объединяющей HTML-разметку, CSS-оформление и JavaScript-поведение в одном файле. Изучены семантические элементы HTML, CSS-переменные, сеточные модели Flexbox и Grid, псевдоэлементы, анимационные переходы и медиазапросы.

Разбор JavaScript показал, как можно реализовать переключение экранов по хешу URL, обрабатывать пользовательский ввод, выполнять арифметические вычисления, измерять время кругов, формировать историю результатов и пересчитывать демонстрационные параметры гоночной стратегии. Страница демонстрирует базовые принципы создания интерактивного одностраничного приложения без сторонних библиотек.

Поставленная цель достигнута: основные блоки исходного кода описаны, назначение ключевых функций и обработчиков событий раскрыто, а также отмечены ограничения демонстрационной модели. В дальнейшем проект можно расширить сохранением данных, более подробной валидацией ввода, тестированием и использованием реальных исходных данных для стратегических расчётов.